

大数减法

那么循环条件是什么呢？还是算出来的

有了大数加法的基础，接着学习大数减法思路是差不多的。

对于加法来说，两个数字相加不论顺序其结果都是一样的。比如： $0+1=1$ $1+0=1$ 加法交换律对吧，小学学过的。

那么对于减法来说就不是这样了，当被减数 $\geq$ 减数计算的差是一个 ≥ 0 的数字；而被减数 $\leq$ 减数计算的差是 ≤ 0 的数字。比如： $2-1=1$ $1-2=-1$

$19-82$ 得到的结果其实就是 $82-19$ 得到的结果，前面多了一个负号。所以，在做减法的时候可以一直用大的数字减去小的数字，如果是小的数字-大的数字那么就在结果的前面写上负号。

所以这里做个判断，看s1和s2存储中哪个数字大。如果s1存放的数字 $>$ s2存放的数字直接s1-s2就好了

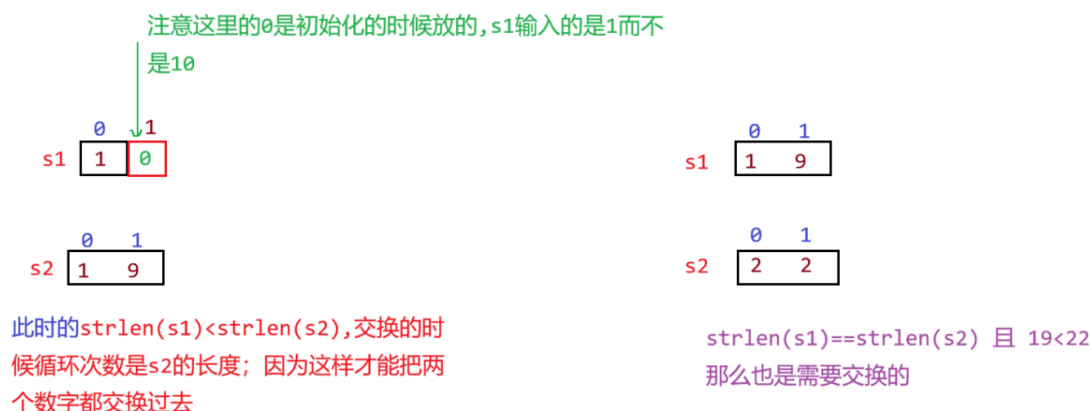
反之就把s1每个元素和s2每个元素交换。

交换条件：

判断s1,s2的长度 最长的字符串肯定是大于较短的字符串的（因为位数比较多）那么

`strlen(s1)==strlen(s2)` 的时候也是需要发生交换的，但是要分情况。（只有s1存放的数字小于s2存放的数字时才需要交换，所以还需要比较一个两个字符的大小关系）如图1所示

通过 `strcmp` 函数来比较出两个字符串大小关系



但是, 在交换的时候循环次数还是遍历s2的长度, 因为对于`strlen(s1)<strlen(s2)`来说, 如果去遍历s1的话, 遇到初始化的0就停止了

图1

当s1<s2计算的结果是负数，所以还需要在其结果前添加负号，这里用 `flag` 表示正负号

```

1 // 交换
2 void swap(char* src1, char* src2)
3 {
4     while (*src2)
5     {
6         char ch = *src1;
7         *src1 = *src2;
8         *src2 = ch;
9
10        src1++;
11        src2++;
12    }
13 }
14
15 int main()
16 {
17     char s1[200] = { 0 };
18     char s2[200] = { 0 };
19
20     char a[200] = { 0 }; // 被减数
21     char b[200] = { 0 }; // 减数
22     char c[200] = { 0 }; // 差
23
24     scanf("%s %s", s1, s2);
25
26     char flag = '+';
27
28     // 判断是否需要交换
29     if (strlen(s1) < strlen(s2) || (strlen(s1) == strlen(s2) && strcmp(s1,
30 s2)))
31     {
32         // 交换
33         swap(s1, s2);
34         flag = '-';
35     }
36     return 0;

```

接着把s1,s2每个元素都转为整数后分别存放数组a和数组b中，这一步的思路和大数加法时一模一样的

```

1 // 倒序存放
2 void reverse(char* src, int* arr)
3 {
4     for (int i = 0; i < strlen(src); i++)

```

```

5  {
6      arr[i] = src[strlen(src) - 1 - i] - '0'; // 转为整数存放
7  }
8  }

```

紧接着就是减法的运算了。

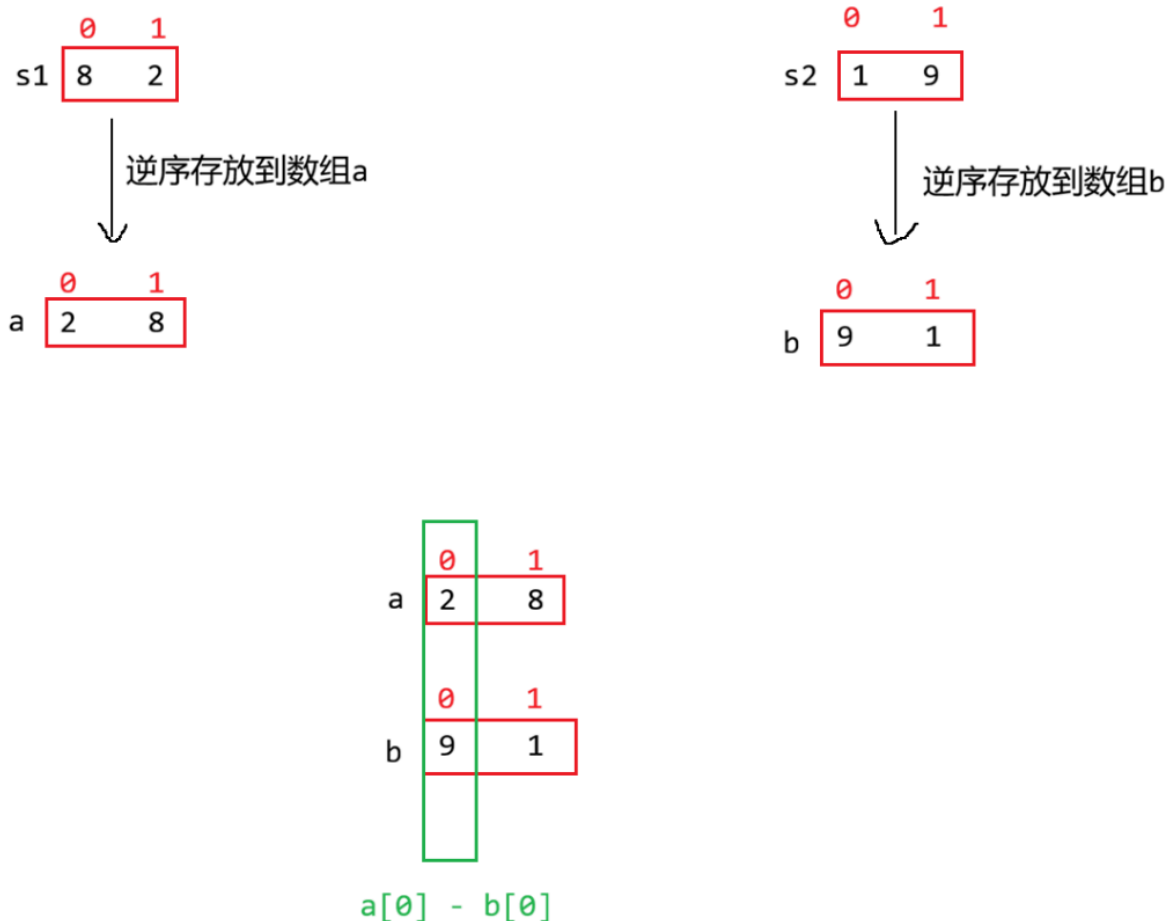


图2

如图2所示，转为整数逆序存放对应的数组中然后来做减法 `a[0]-b[0]` 得到是-7，所以不能直接减这里需要判断是否需要向右一个元素 `a[i+1]` 借1“当10”然后再加自身；2向前借1当10再上自身变成12-9也就是：`a[i] = a[i]+10`；那么 `a[i+1]` 元素则需要自减1，因为前面借走了1

循环条件是什么呢？其实前面把最大的数字存到了s1中，所以只需要遍历s1的长度就好了

```

1 void sub(char* src, int* arr1, int* arr2, int* ret)
2 {
3     for (int i = 0; i < strlen(src); i++)
4     {
5         // 判断是否借位
6         if (arr1[i] < arr2[i])
7         {

```

```

8          // 向前借1
9          arr1[i] = arr1[i] + 10;
10         arr1[i + 1]--;
11     }
12     ret[i] = arr1[i] - arr2[i];
13 }
14 }

```

最后，逆序输出结果。在输出结果前还需要判断当前是否为负数，也就是前面的 `flag`；数组c的大小是210不能从最后一位倒着往前输出这显然不对，所以还需要找到逆序输出的其实位置。

两个数相减其结果绝对不会超过两个数中最大的数字的位数！如图3所示。

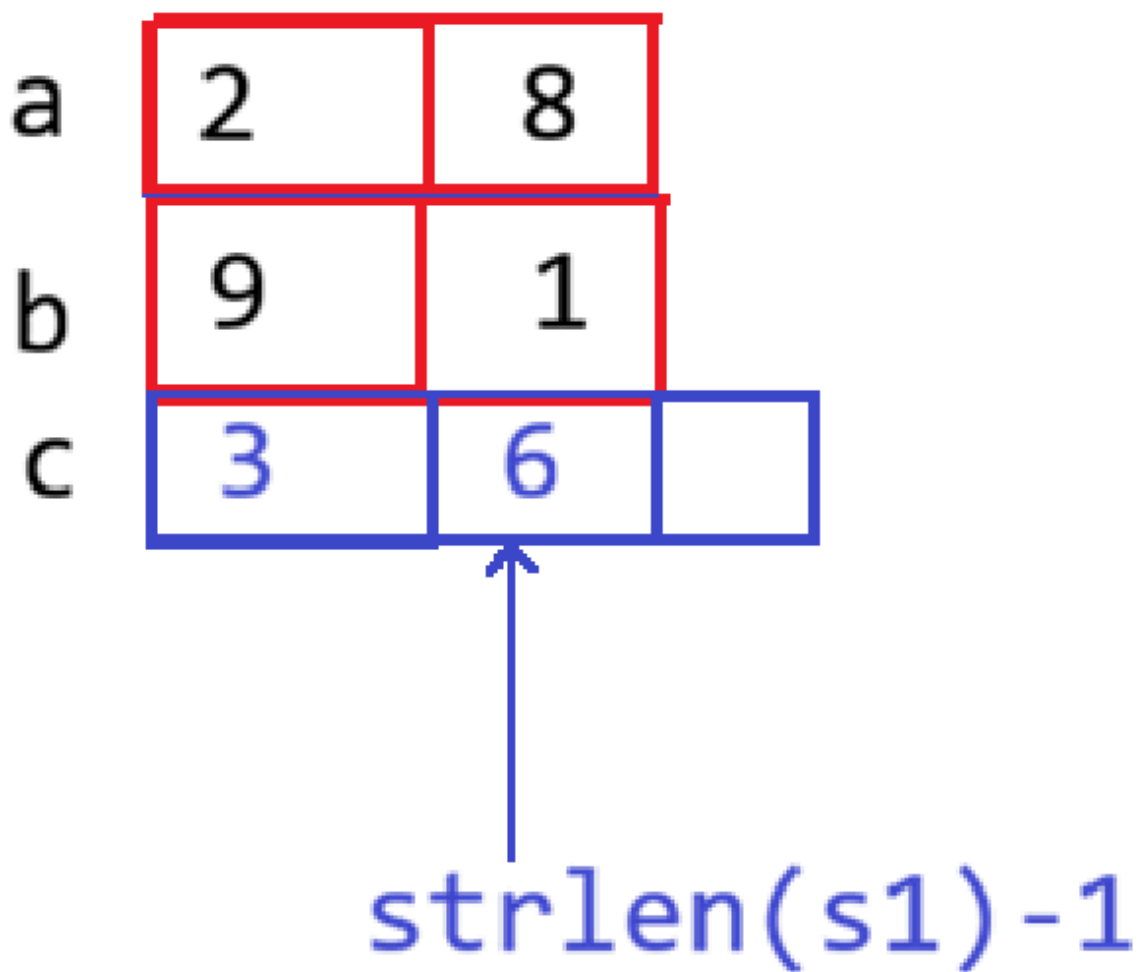


图3

```

1 void print(char* dep, char* src, int* ret)
2 {
3     // 判断是否负数
4     if (*dep == '-')

```

```

5     {
6         printf("%c", *dep);
7     }
8
9     // 找到倒序输出的起始位置
10    int index = 0;
11    for (int i = (int)strlen(src)-1; i >= 0 ; i--)
12    {
13        if (ret[i] != 0)
14        {
15            // 记录下该位置
16            index = i;
17            break;
18        }
19    }
20
21
22    for (int i = index; i >= 0; i--)
23    {
24        printf("%d", ret[i]);
25    }
26
27 }

```

完整代码：

```

▼
1 #define _CRT_SECURE_NO_WARNINGS 1
2
3 #include <stdio.h>
4 #include <string.h>
5
6 // 交换
7 void swap(char* src1, char* src2)
8 {
9     while (*src2)
10    {
11        char ch = *src1;
12        *src1 = *src2;
13        *src2 = ch;
14
15        src1++;
16        src2++;
17    }
18 }
19
20

```

```

21 // 倒序存放
22 void reverse(char* src, int* arr)
23 {
24     for (int i = 0; i < strlen(src); i++)
25     {
26         arr[i] = src[strlen(src) - 1 - i] - '0'; // 转为整数存放
27     }
28 }
29
30 void sub(char* src, int* arr1, int* arr2, int* ret)
31 {
32     for (int i = 0; i < strlen(src); i++)
33     {
34         // 判断是否借位
35         if (arr1[i] < arr2[i])
36         {
37             // 向前借1
38             arr1[i] = arr1[i] + 10;
39             arr1[i + 1]--;
40         }
41         ret[i] = arr1[i] - arr2[i];
42     }
43 }
44
45 void print(char* dep, char* src, int* ret)
46 {
47     // 判断是否负数
48     if (*dep == '-')
49     {
50         printf("%c", *dep);
51     }
52
53     // 找到倒序输出的起始位置
54     int index = 0;
55     for (int i = (int)strlen(src)-1; i >= 0 ; i--)
56     {
57         if (ret[i] != 0)
58         {
59             // 记录下该位置
60             index = i;
61             break;
62         }
63     }
64
65     for (int i = index; i >= 0; i--)
66     {
67         printf("%d", ret[i]);

```

```
69     }
70
71 }
72
73
74 int main()
75 {
76     char s1[200] = { 0 };
77     char s2[200] = { 0 };
78
79     int a[210] = { 0 }; // 被减数
80     int b[210] = { 0 }; // 减数
81     int c[210] = { 0 }; // 差
82
83     scanf("%s %s", s1, s2);
84
85     char flag = '+';
86
87     // 判断是否需要交换
88     if (strlen(s1) < strlen(s2) || (strlen(s1) == strlen(s2) && (strcmp(s1,
89 s2) < 0)))
90     {
91         // 交换
92         swap(s1, s2);
93         flag = '-';
94     }
95
96     // 倒序存放
97     reverse(s1, a);
98     reverse(s2, b);
99
100    // 减法运算
101    sub(s1, a, b, c);
102
103    // 输出结果
104    print(&flag, s1, c);
105
106    return 0;
107 }
```